RTOS RTC
开发指南

版本号: 1.11

发布日期: 2025.09.17

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.7.16	AWA1637	初始版本
1.1	2021.4.27	AWA1637	增加 F133 平台说明
1.2	2022.07.30	XAA0190	增加 R128 说明
1.3	2023.04.23	XAA0190	1.3 节增加 MR527 说明
1.4	2023.08.21	XAA0190	1.3 节增加 XR875 说明
1.5	2023.12.08	XAA0190	章节 2 和章节 3 修改部分描述信息
1.6	2024.01.18	XAA0190	章节 2 和章节 3 细化内容
1.7	2024.02.26	XAA0309	1.3 节增加 AI985 说明
1.8	2024.08.22	XAA0309	1.3 节增加 MR536 说明
1.9	2024.10.26	AWA1427	基于新模板更新文档
1.1	2025.04.24	XAA0339	更新文档格式
1.1	2025.04.24	XAA0339	更新文档格式
1.11	2025.09.17	XAA0331	新增 V861 说明

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 1

1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 2
1.4.1 标志说明 2
1.4.2 地址与数据描述方法约定 2
1.5 相关术语介绍 3
1.5.1 硬件术语 3
1.5.2 软件术语 3
1.6 模块功能 3
1.7 模块配置 4
1.7.1 menuconfig 配置说明 4
1.7.2 RTC 时钟源配置 4
1.7.2.1 大小核平台 4
1.7.2.2 R128/XR875 平台 4
1.8 源码结构 4
1.9 软件框架 6

2 模块接口说明 7

2.1 数据结构 8
2.1.1 RTC 时间结构体 8
2.1.2 RTC 闹钟结构体 8
2.2 hal_rtc_init 8
2.3 hal_rtc_deinit 9
2.4 hal_rtc_gettime 9
2.5 hal_rtc_settime 9
2.6 hal_rtc_getalarm 10
2.7 hal_rtc_setalarm 10
2.8 hal_rtc_alarm_irq_enable 10
2.9 hal_rtc_register_callback 10
2.10 hal_rtc_set_fel_flag 11
2.11 hal_rtc_probe_fel_flag 11
2.12 hal_rtc_clear_fel_flag 11
2.13 hal_rtc_get_bootmode_flag 11
2.14 hal_rtc_set_bootmode_flag 12
2.15 hal_rtc_write_data 12
2.16 hal_rtc_read_data 12

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.17 hal_rtc_min_year_show 13
2.18 hal_rtc_max_year_show 13

3 功能开发 14
3.1 功能概述 14
3.2 开发流程 14
3.2.1 设置 RTC 时间 14
3.2.2 获取 RTC 时间 15
3.2.3 设置以时间戳的方式设置 RTC 时间 15
3.2.4 获取 RTC 时间，以时间戳方式返回 15
3.2.5 设置 RTC 闹钟 15
3.3 编程示例 16

4 调试方法 18
4.1 调试信息 18
4.2 调试工具 18

5 FAQ 19
5.1 RTC 时间范围 19
5.2 XR875S1 C906 RTC 闹钟中断未触发 19
5.2.1 问题现象 19
5.2.2 问题原因 19
5.2.3 解决方法 20
5.3 一号通 FAQ 列表 20

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文档介绍 RTOS 中 RTC 驱动的接口及使用方法。

1.2 目标读者

- RTC 驱动层开发人员
- RTC 应用层开发人员
- RTC 使用人员
- RTC 模块维护人员

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V459	Melis	hal_rtc.c
F133	Melis	hal_rtc.c
F135	Melis	hal_rtc.c
F136	Melis	hal_rtc.c
R328	FreeRTOS	hal_rtc.c
R128	FreeRTOS	hal_rtc.c
MR527	FreeRTOS	hal_rtc.c
XR875	FreeRTOS	hal_rtc.c
AI985	FreeRTOS	hal_rtc.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

产品名称	内核版本	驱动文件
MR536	FreeRTOS	hal_rtc.c
V821	FreeRTOS	hal_rtc.c
MR153	FreeRTOS	hal_rtc.c
T153	FreeRTOS	hal_rtc.c
V861	FreeRTOS	hal_rtc.c

1.4 文档约定

1.4.1 标志说明

注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

:::note 为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)
X	00X，XX1	数据描述中，X 代表 0 或 1

例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.5 相关术语介绍

1.5.1 硬件术语

表 1-3: 硬件术语

术语	解释说明
RTC	Real-Time Clock, 实时时钟

1.5.2 软件术语

表 1-4: 软件术语

术语	解释说明
HAL	Hardware Abstraction Layer，硬件抽象层

1.6 模块功能

实时时钟 RTC 用于实现时钟计数和定时唤醒功能，能够实时显示日，时，分，秒功能。该模块电源独立，可以在系统掉电后工作，RTC 具有以下特点。

- 内部具有一个 16bit 的日计数器，5bit 的小时计数器，6bit 的分计数器，6bit 的秒计数器
- 可外接 32768Hz 低频晶振作为计数时钟
- 可随时软件配置初始值
- 具有定时闹钟功能，可产生中断及唤醒外围设备
- 8 个用户寄存器可存放掉电信息
- 多个特殊寄存器记录 BROM 相关信息

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.7 模块配置

1.7.1 menuconfig 配置说明

```
Drivers Options  --->
  soc related device drivers  --->
    RTC Devices  --->
      [*] enable rtc driver   #使能RTC驱动
      [ ]  enable rtc hal APIs test command   #RTC驱动测试代码
```

1.7.2 RTC 时钟源配置

RTC 模块支持配置使用内部 32K 还是外部 32K 时钟源。如果对时间精度比较敏感，建议选择外部 32K 时钟源。

1.7.2.1 大小核平台

RTC 时钟源的选择一般在 Linux 配置，RTOS 不需要配置。

1.7.2.2 R128/XR875 平台

在 M33 RTOS 配置：CONFIG_USE_EXTERNAL_LOW_FREQ_CRYSTAL_CLK

```
Architecture Options  --->
  [*] Use the external low frequency crystal clock
```

1.8 源码结构

1. 驱动源码

```
hal/source/rtc/
├──── hal_rtc.c
├──── Kconfig
├──── Makefile
├──── platform
│    ├──── rtc_sun20iw1.h
│    ├──── rtc_sun20iw2.h
│    ├──── rtc_sun50iw11.h
│    ├──── rtc_sun55iw3.h
│    ├──── rtc_sun8iw18.h
│    └──── rtc_sun8iw19.h
└    ...
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
├── platform_rtc.h
└── rtc-lib.c
```

RTC 模块寄存器的基本配置位于文件 rtc_{platform}.h 里面，包括每个 RTC 的寄存器地址和中断号，部分配置如下：

```
#寄存器基址
#define SUNXI_RTC_BASE     0x07000000
#define SUNXI_RTC_DATA_BASE    (SUNXI_RTC_BASE+0x100)

#中断
#define SUNXI_GIC_START 32
#define SUXNI_IRQ_RTC (SUNXI_GIC_START + 105)
```

2. 驱动 APIs 声明头文件

```
include/hal/
└── sunxi_hal_rtc.h
```

3. 驱动 APIs 测试代码

```
hal/test/rtc/
├── Makefile
└── test_rtc.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.9 软件框架

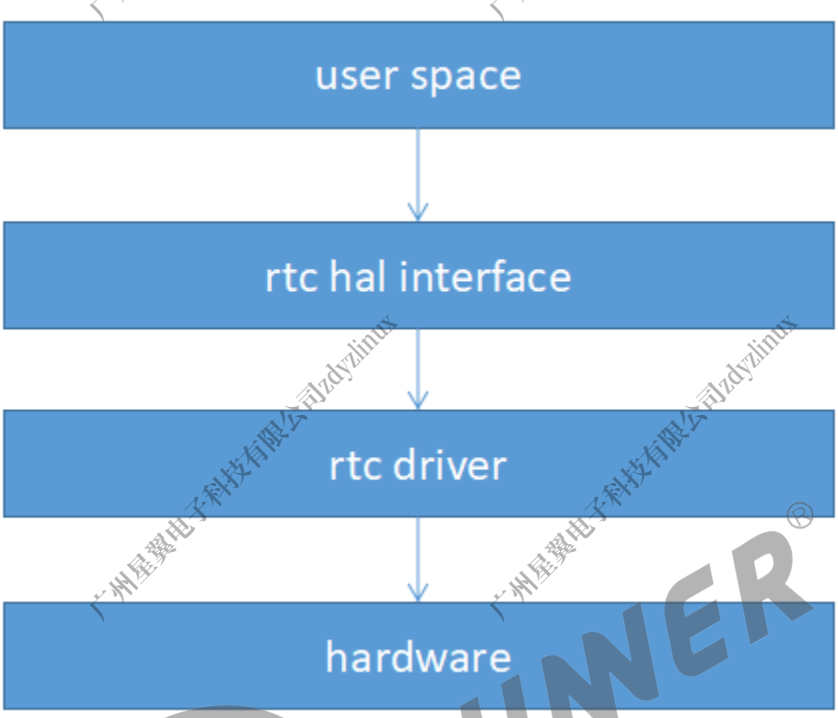

图 1-1: RTOS RTC 框架

RTC 模块提供给用户使用的主要包括设置/读取时间，设置/读取闹钟，以及设置闹钟回调接口等等。

主要框架如上图所示：用户空间调用 rtc hal interface(具体详见模块接口说明章节) 实现时间的设置与读取，闹钟的设置与关闭

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块接口说明

表 2-1: 模块接口列表

API	解释说明
hal_rtc_init	RTC 模块初始化
hal_rtc_deinit	RTC 模块反初始化
hal_rtc_gettime	获取 RTC 时间
hal_rtc_settime	设置 RTC 时间
hal_rtc_getalarm	获取 RTC 闹钟数据
hal_rtc_setalarm	获取 RTC 闹钟
hal_rtc_alarm_irq_enable	使能闹钟中断
hal_rtc_register_callback	注册闹钟回调接口
rtc_time64_to_tm	将 time64_t 时间戳转为驱动的 rtc_time 类型
rtc_tm_to_time64	将 rtc_time 类型转为 time64_t 时间戳
rtc_month_days	获取某年某月有多少天
rtc_year_days	获取某年某月某日距离元旦过了多少天
hal_rtc_set_fel_flag	设置 FEL 标志位
hal_rtc_probe_fel_flag	探测 FEL 标志位并返回其值
hal_rtc_clear_fel_flag	清除 FEL 标志位
hal_rtc_write_data	在指定索引位置向 RTC 写入数据
hal_rtc_read_data	从指定索引位置从 RTC 读取数据
hal_rtc_min_year_show	将 RTC 支持的最小年份值保存到传入的指针所指向的变量中
hal_rtc_max_year_show	将 RTC 支持的最大年份值保存到传入的指针所指向的变量中
hal_rtc_get_bootmode_flag	获取当前的启动模式标志
hal_rtc_set_bootmode_flag	设置启动模式标志为指定的值

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.1 数据结构

由于 GPIO 需要配置每个引脚的引脚复用功能，中断类型，驱动能力，上下拉，输出/输入数据，输入/输出方向等等，所以对 GPIO 的这些配置都封装在一个 enum 枚举结构里面，方便使用。下面是一些配置的定义，想要了解更多的数据结构可以到 hal_gpio.h 查看。

2.1.1 RTC 时间结构体

该结构体用来保存 RTC 模块的时间，具体如下所示：

```
struct rtc_time
{
    int tm_sec; //秒
    int tm_min; //分
    int tm_hour; //时
    int tm_mday; //天
    int tm_mon; //月
    int tm_year; //年
    int tm_wday;
    int tm_yday;
    int tm_isdst;
};
```

说明

1. **rtc_time 的 tm_year 值是相对 1900 年的偏移.**
2. **rtc_time 的 tm_mon 取值范围是 0~11.**

2.1.2 RTC 闹钟结构体

该结构体用来保存 RTC 模块的闹钟时间，具体如下所示：

```
struct rtc_wkalrm
{
    unsigned char enabled;  /* 0 = alarm disabled, 1 = alarm enabled */
    unsigned char pending;  /* 0 = alarm not pending, 1 = alarm pending */
    struct rtc_time time;   /* time the alarm is set to */
};
```

说明

1. **rtc_wkalrm 中的 rtc_time 年份值是相对 1900 年的偏移.**
2. **rtc_wkalrm 中的 rtc_time 月份范围是 0~11.**

2.2 hal_rtc_init

- 作用：初始化 RTC 模块

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 参数：
  - void
- 返回：
  - -1: 失败
  - 0: 成功

2.3 hal_rtc_deinit

- 作用：反初始化 RTC 模块
- 参数：
  - void
- 返回：
  - -1: 失败
  - 0: 成功

2.4 hal_rtc_gettime

- 作用：获取时间值，保存在 rtc_tm 结构体里面
- 参数：
  - rtc_tm: 时间结构体指针
- 返回：
  - -1: 失败
  - 0: 成功

2.5 hal_rtc_settime

- 作用：设置时间值
- 参数：
  - rtc_tm，设置时间结构体指针
- 返回值：
  - -1: 失败
  - 0: 成功

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.6 hal_rtc_getalarm

- 作用：获取闹钟数据
- 参数：
  - wkalrm，保存闹钟数据的结构体
- 返回值：
  - -1: 失败
  - 0: 成功

2.7 hal_rtc_setalarm

- 作用：设置闹钟数据
- 参数：
  - wkalrm，保存闹钟数据的结构体指针
- 返回值：
  - -1: 失败
  - 0: 成功

2.8 hal_rtc_alarm_irq_enable

- 作用：使能闹钟中断
- 参数：
  - enable：1，使能中断，0 失能中断
- 返回值：
  - -1: 失败
  - 0: 成功

2.9 hal_rtc_register_callback

- 功能：注册闹钟回调接口
- 参数：
  - user_callback：回调函数指针

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回值：
  - -1: 失败
  - 0: 成功

2.10 hal_rtc_set_fel_flag

- 功能：设置 FEL 标志位
- 参数：
  - void
- 返回值：
  - void

2.11 hal_rtc_probe_fel_flag

- 功能：探测 FEL 标志位并返回其值
- 参数：
  - void
- 返回值：
  - FEL 标志位的值

2.12 hal_rtc_clear_fel_flag

- 功能：清除 FEL 标志位
- 参数：
  - void
- 返回值：
  - void

2.13 hal_rtc_get_bootmode_flag

- 功能：获取当前的启动模式标志
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - void
- 返回值：
  - 启动模式标志的值
  - -1: 失败

2.14 hal_rtc_set_bootmode_flag

- 功能：设置启动模式标志为指定的值
- 参数：
  - flag: 要设置的启动模式标志值
- 返回值：
  - 0: 成功
  - -1: 失败

2.15 hal_rtc_write_data

- 功能：在指定索引位置向 RTC 写入数据
- 参数：
  - index: 写入数据的索引位置
  - val: 要写入的数据
- 返回值：
  - void

2.16 hal_rtc_read_data

- 功能：从指定索引位置从 RTC 读取数据
- 参数：
  - index: 写入数据的索引位置
- 返回值：
  - 读取到的数据

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.17 hal_rtc_min_year_show

- 功能：将 RTC 支持的最小年份值保存到传入的指针所指向的变量中
- 参数：
  - min: 用于保存最小年份值的无符号整型指针
- 返回值：
  - void

2.18 hal_rtc_max_year_show

- 功能：将 RTC 支持的最大年份值保存到传入的指针所指向的变量中
- 参数：
  - max: 用于保存最大年份值的无符号整型指针
- 返回值：
  - voids

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 功能开发

3.1 功能概述

RTC 驱动提供了以下常用功能：

- 设置时间值
- 获取时间值
- 设置闹钟

3.2 开发流程

3.2.1 设置 RTC 时间

步骤 1 定义 rtc_time 结构体变量，填充时间信息。

说明

1. **rtc_time 的 tm_year 值是相对 1900 年的偏移，例如设置年份为 2020 年，那么 tm_year 应该赋值 2020 - 1900 = 120.**
2. **rtc_time 的 tm_mon 取值范围是 0~11，相比实际月份少 1。例如设置月份为 5 月，那么 tm_mon 应该赋值 5 - 1 = 4.**

例如：设置 RTC 时间为 2024 年 10 月 20 日 15 时 30 分 00 秒

```
struct rtc_time rtc_tm;

rtc_tm.tm_year = 2024 - 1900;
rtc_tm.tm_mon = 10 - 1 ;
rtc_tm.tm_mday = 20;
rtc_tm.tm_hour = 15;
rtc_tm.tm_min = 30;
rtc_tm.tm_sec = 00;
```

注意

- **tm_year 和 tm_mon 需要转换**

步骤 2 调用 hal_rtc_settime 设置时间。

```
if (hal_rtc_settime(&rtc_tm))
{
    printf("sunxi rtc settime error\n");
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
}
```

3.2.2 获取 RTC 时间

步骤 1 定义 rtc_time 结构体变量。

步骤 2 调用 hal_rtc_gettime 获取。

⚠ 注意

- **获取到的 rtc_time 里面 tm_year 和 tm_mon 注意转换**

3.2.3 设置以时间戳的方式设置 RTC 时间

步骤 1 定义 rtc_time 结构体变量。

步骤 2 调用 rtc_time64_to_tm 将时间戳转为 rtc_time 类型。

```
uint64_t stamp = xxxxx;
struct rtc_time time;
rtc_time64_to_tm(stamp, &time);
```

步骤 3 调用 hal_rtc_settime 设置时间。

3.2.4 获取 RTC 时间，以时间戳方式返回

步骤 1 定义 rtc_time 结构体变量。

步骤 2 调用 hal_rtc_gettime 获取。

步骤 3 调用 rtc_tm_to_time64 转换单位并返回。

3.2.5 设置 RTC 闹钟

步骤 1 定义 rtc_wkalrm 结构体变量，填充闹钟信息。

例如：设置 RTC 闹钟为 2024 年 10 月 20 日 18 时 25 分 30 秒

```
struct rtc_wkalrm wkalrm;

wkalrm.time.tm_year = 2024 - 1900;
wkalrm.time.tm_mon = 10 - 1;
wkalrm.time.tm_mday = 20;
wkalrm.time.tm_hour = 18;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
wkalrm.time.tm_min = 25;
wkalrm.time.tm_sec = 30;
```

⚠ 注意

- **tm_year 和 tm_mon 需要转换**

步骤 2 调用 hal_rtc_register_callback 注册回调函数。

步骤 3 调用 hal_rtc_setalarm 设置闹钟。

```
  if (hal_rtc_getalarm(&wkalrm))
  {
    printf("sunxi rtc getalarm error\n");
  }
```

步骤 4 调用 hal_rtc_alarm_irq_enable 使能闹钟中断。

3.3 编程示例

1. 驱动测试示例

路径：lichee/rtos-hal/hal/test/rtc/test_rtc.c

```
static int callback(void)
{
  printf("alarm interrupt\n");
  return 0;
}

static int cmd_test_rtc(int argc, const char **argv)
{
  unsigned int enable = 1;
  struct rtc_time rtc_tm;
  struct rtc_wkalrm wkalrm;

  // 初始化RTC 驱动
  hal_rtc_init();

  // 注册RTC闹钟回调函数
  hal_rtc_register_callback(callback);

  // 获取当前时间
  if (hal_rtc_gettime(&rtc_tm))
  {
    printf("sunxi rtc gettime error\n");
  }

  wkalrm.enabled = 1;
  // 将获取到的时间赋值给闹钟时间，并对时间做细微调整
  wkalrm.time = rtc_tm;
  if(rtc_tm.tm_min > 0)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    rtc_tm.tm_min -= 1;
  else
    wkalrm.time.tm_min += 1;

  printf("alarm time %04d-%02d-%02d %02d:%02d:%02d\n",
      wkalrm.time.tm_year + 1900, wkalrm.time.tm_mon + 1, wkalrm.time.tm_mday,
      wkalrm.time.tm_hour, wkalrm.time.tm_min, wkalrm.time.tm_sec);

  // 重新设置时间
  if (hal_rtc_settime(&rtc_tm))
  {
    printf("sunxi rtc settime error\n");
  }

  // 设置闹钟
  if (hal_rtc_setalarm(&wkalrm))
  {
    printf("sunxi rtc setalarm error\n");
  }

  if (hal_rtc_getalarm(&wkalrm))
  {
    printf("sunxi rtc getalarm error\n");
  }

  if (hal_rtc_gettime(&rtc_tm))
  {
    printf("sunxi rtc gettime error\n");
  }

  // 使能闹钟中断
  hal_rtc_alarm_irq_enable(enable);

  return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 调试方法

4.1 调试信息

RTC 驱动默认输出 DEBUG 信息：

路径：lichee/rtos/drivers/rtos-hal/hal/source/rtc/hal_rtc.c

```
#define CONFIG_DRIVERS_RTC_DEBUG
#ifdef CONFIG_DRIVERS_RTC_DEBUG
#define RTC_INFO(fmt, arg...) hal_log_info(fmt, ##arg)
#else
#define RTC_INFO(fmt, arg...) do {}while(0)
#endif
```

4.2 调试工具

- 配置 CONFIG_HAL_TEST_RTC 后，可以使用 hal_rtc 命令验证 RTC 模块是否能正常工作。

```
c906>hal_rtc
Read hardware RTC time 1970-01-01 03:43:36
alarm time 1970-01-01 03:43:36
Will set hardware RTC time 0000-01-01 03:42:36
Read hardware RTC time 1970-01-01 03:42:36
set alarm seconds:60 enable:1
 alarm_cnt: 60, alarm_cur: 0
Read hardware RTC time 1970-01-01 03:42:36
alarm_seconds: 13416
Read hardware RTC time 1970-01-01 03:42:36

c906>alarm interrupt  //1分钟后，触发闹钟
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 FAQ

5.1 RTC 时间范围

RTC 模块记录的有效时间范围为 1900-2097 年。

5.2 XR875S1 C906 RTC 闹钟中断未触发

5.2.1 问题现象

客户在 C906 上初始化了 RTC，并正确设置了 RTC 闹钟，注册了闹钟回调函数。

但闹钟时间到了回调未正常执行。

5.2.2 问题原因

因为 M33 RTOS 默认配置了 RTC 驱动，在系统初始化的时候，把 RTC 初始化了：

路径：lichee/rtos/arch/arm/armv8m/sun20iw2p1/sun20i.c

```
static void prvSetupHardware( void )
{
 ...
#ifdef CONFIG_DRIVERS_RTC
   hal_rtc_init();
#endif
 ...
}
```

在 hal_rtc_init 函数中，会注册 RTC 的中断。

```
   rtc_dev->irq = SUXNI_IRQ_RTC;

   ret = hal_request_irq(rtc_dev->irq, hal_rtc_alarmirq, "rtc", rtc_dev);
   if (ret < 0)
   {
      RTC_ERR("Could not request IRQ");
      return -1;
   }
   hal_enable_irq(rtc_dev->irq);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

C906 和 M33 两个核的 SUXNI_IRQ_RTC 中断号是不一样的，两边都注册的了的话，中断 Pending 会先被 M33 处理，导致 C906 没法正常触发中断。

5.2.3 解决方法

如果 C906 需要使用 RTC 功能，确保关闭 M33 的 RTC 驱动。

5.3 一号通 FAQ 列表

- FAQ2828

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。